

Guía de Estudio Completa — Proyecto PulmoScan / IA Radiografías

Universidad Popular del Cesar — Inteligencia Artificial — III Corte 2026-I

Integrantes: Mateo López Patiño, Anaclaudia Vega Martínez

Docente: Tonny Enrique Jiménez Márquez

Este documento resume **todo el proyecto** de principio a fin: qué problema resolvimos, qué conceptos de IA usamos, qué hace cada archivo, qué resultados obtuvimos y **por qué** tomamos cada decisión técnica. Está pensado para estudiar y sustentar.

Tabla de contenidos

1. ¿Qué hace este proyecto?
2. El dataset NIH ChestX-ray14
3. Las 14 patologías que detectamos
4. Conceptos fundamentales de IA que debes dominar
5. DenseNet-121 y CheXNet en detalle
6. Transfer Learning explicado paso a paso
7. Fases del proyecto (qué hicimos y en qué orden)
8. Fase 1: Análisis Exploratorio (EDA)
9. Fase 2: Diseño del sistema
10. Fase 3: Entrenamiento en Kaggle
11. Fase 4: Evaluación y resultados
12. Fase 5: App de demostración (Streamlit)
13. Grad-CAM: interpretabilidad del modelo
14. Mapa de archivos del repositorio
15. Flujo de datos completo (diagrama)
16. Resultados numéricos clave (para memorizar)
17. Preguntas frecuentes para sustentación
18. Glosario

1. ¿Qué hace este proyecto?

Problema clínico

Un radiólogo puede tardar varios minutos en revisar una radiografía de tórax y detectar patologías. Nuestro sistema **automatiza esa tarea parcialmente**: dada una radiografía frontal, el modelo devuelve la **probabilidad** de que estén presentes **14 patologías torácicas** distintas.

Tipo de problema en términos de IA

Aspecto	Nuestro caso
Tipo de entrada	Imagen (radiografía PNG en escala de grises)
Tipo de salida	14 probabilidades independientes (0 a 1)
Tipo de clasificación	Multi-etiqueta (multi-label)
Arquitectura	DenseNet-121 con Transfer Learning desde ImageNet
Referencia académica	CheXNet (Stanford, 2017)

Lo que NO es este proyecto

- **No es un diagnóstico médico.** Es una herramienta académica de apoyo; la app muestra un disclaimer.
- **No elige una sola enfermedad** (no es clasificación simple de 14 clases).
- **No reemplaza al radiólogo.** Sugiere dónde mirar y con qué probabilidad hay hallazgos.

2. El dataset NIH ChestX-ray14

Origen

Publicado por el **National Institutes of Health (NIH)** en 2017. Es uno de los datasets públicos más usados en investigación de IA médica para radiografías de tórax.

Números clave

Métrica	Valor
Total de imágenes	112.120 radiografías frontales
Pacientes únicos	30.805
Patologías etiquetadas	14
Tamaño total (imágenes)	~42 GB descomprimido
Resolución promedio	2.646 × 2.486 píxeles
Archivo de metadatos	Data_Entry_2017.csv

Cómo se etiquetaron las imágenes

Las etiquetas **no las puso un radiólogo imagen por imagen**. El NIH usó **Procesamiento de Lenguaje Natural (NLP)** sobre los informes radiológicos escritos por médicos. Extrajeron menciones de patologías y las asociaron a cada imagen.

Implicación: hay ruido en las etiquetas. Por ejemplo, *Infiltration* es ambigua y co-ocurre con muchas otras patologías. Esto explica por qué algunas clases son difíciles de predecir.

Columnas importantes del CSV

Columna original (NIH)	Renombrada	Significado
Image Index	image_id	Nombre del PNG, ej. 00000001_000.png
Finding Labels	labels_raw	Patologías separadas por
Patient ID	patient_id	ID anónimo del paciente
Patient Age	age	Edad
Patient Gender	gender	M o F
View Position	view	PA (de pie) o AP (acostado)
Follow-up #	followup	Número de visita del paciente (0 = primera)

Splits oficiales del NIH

El NIH publicó dos archivos de texto:

- `train_val_list.txt` → **86.524 imágenes** (28.008 pacientes) para entrenar y validar
- `test_list.txt` → **25.596 imágenes** (2.797 pacientes) solo para evaluación final

Verificación crítica: la intersección de pacientes entre train_val y test es **0**. No hay *data leakage* entre conjuntos oficiales.

Nuestros splits finales (después de `build_splits.py`)

Split	Imágenes	Pacientes	Uso
train	69.219	22.407	Entrenar el modelo
val	17.305	5.601	Elegir el mejor checkpoint
test	25.596	2.797	Evaluación final (una sola vez)

3. Las 14 patologías que detectamos

#	Patología	Qué es (en simple)	Prevalencia aprox.
1	Atelectasis	Colapso parcial del pulmón	10,3%
2	Cardiomegaly	Corazón agrandado	2,5%
3	Effusion	Líquido en el espacio pleural (derrame)	11,9%
4	Infiltration	Opacidades difusas en el pulmón	17,7% (la más común)
5	Mass	Lesión sólida > 3 cm	5,2%
6	Nodule	Lesión redondeada < 3 cm	5,7%
7	Pneumonia	Infección pulmonar	1,3%
8	Pneumothorax	Aire en el espacio pleural (pulmón colapsado)	4,7%

#	Patología	Qué es (en simple)	Prevalencia aprox.
9	Consolidation	Pulmón relleno de fluido/tejido sólido	4,2%
10	Edema	Líquido en el tejido pulmonar	2,1%
11	Emphysema	Destrucción de alvéolos (fumadores)	2,2%
12	Fibrosis	Cicatrización del tejido pulmonar	1,5%
13	Pleural_Thickening	Engrosamiento de la pleura	3,0%
14	Hernia	Órganos abdominales suben al tórax	0,2% (la más rara)

Además, el dataset tiene la etiqueta "**No Finding**" (sin hallazgo): ~53,8% de las imágenes. En nuestro modelo **no es una clase de salida**. Se representa implícitamente cuando las 14 probabilidades son bajas.

4. Conceptos fundamentales de IA que debes dominar

4.1. Red Neuronal Convolutacional (CNN)

Una **CNN** es un tipo de red neuronal diseñada para procesar **imágenes**. En lugar de conectar cada píxel con cada neurona (imposible en imágenes grandes), usa **filtros convolucionales** que detectan patrones locales:

- Bordes y texturas en capas tempranas
- Formas y estructuras en capas medias
- Objetos y regiones semánticas en capas profundas

En nuestro proyecto: DenseNet-121 es una CNN de 121 capas que actúa como "extractor de características" de la radiografía.

4.2. Clasificación multi-etiqueta vs multi-clase

	Multi-clase (softmax)	Multi-etiqueta (sigmoid) — nuestro caso
Pregunta	¿Cuál de estas 14 enfermedades tiene?	¿Tiene cada una de las 14 enfermedades?
Salidas	14 probabilidades que suman 1	14 probabilidades independientes (0 a 1 cada una)
Activación	Softmax	Sigmoid (una por clase)
Pérdida	Cross-Entropy	Binary Cross-Entropy (BCE)
Ejemplo	Solo "Pneumonia"	"Cardiomegaly + Effusion + Infiltration" a la vez

Por qué multi-etiqueta: el 22% de las imágenes con hallazgos tienen **2 o más patologías simultáneas**. Una radiografía puede ser neumonía + efusión + atelectasia al mismo tiempo.

4.3. Binary Cross-Entropy (BCE) y BCEWithLogitsLoss

BCE mide qué tan lejos está cada predicción de la etiqueta real (0 o 1) para cada patología:

$$\text{BCE} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)]$$

BCEWithLogitsLoss en PyTorch combina **sigmoid + BCE** en un solo paso numéricamente estable. El modelo devuelve **logits** (valores crudos sin sigmoid); la pérdida aplica sigmoid internamente.

4.4. pos_weight — compensar desbalance de clases

Desbalance: Infiltration tiene 19.894 casos; Hernia solo 227 (**87 veces más**). Sin compensación, el modelo aprende a decir siempre "no hay Hernia" y obtiene 99,8% de accuracy en esa clase, pero es inútil clínicamente.

Solución: **pos_weight** en BCEWithLogitsLoss:

$$\text{pos_weight}_i = \frac{\#\text{negativos}_i}{\#\text{positivos}_i}$$

Patología	pos_weight aprox. (train)
Hernia	~626
Pneumonia	~94
Infiltration	~4,6

Un falso negativo en Hernia se penaliza **626 veces más** que un falso positivo. Así el modelo aprende también las clases raras.

4.5. Data Leakage (fuga de datos)

Ocurre cuando información del conjunto de **test** "se cuela" al entrenamiento, inflando métricas artificialmente.

En medicina, el leakage más peligroso es por paciente:

- Un paciente puede tener hasta **184 radiografías** (seguimientos).
- Si repartimos imágenes al azar, las costillas del paciente 13118 estarían en train Y en test.
- El modelo memoriza al paciente, no aprende a diagnosticar.

Nuestra solución: dividir siempre por **patient_id** usando **GroupKFold**, nunca por imagen.

4.6. Data Augmentation (aumento de datos)

Técnicas para **crear variaciones artísticas** de las imágenes de entrenamiento y hacer el modelo más robusto:

Transformación	Valor en nuestro proyecto	¿Por qué?
Rotación	$\pm 7^\circ$	Pequeñas variaciones de inclinación
Traslación	$\pm 5\%$	Paciente ligeramente descentrado
Escala	95%–105%	Variaciones de zoom
Brillo/contraste	$\pm 10\%$	Diferentes equipos de rayos X
Flip horizontal	NO	Cambiaría la lateralidad cardíaca (corazón a la izquierda)

Solo se aplica en **train**. Val y test usan solo resize + normalización.

4.7. Overfitting (sobreajuste)

El modelo memoriza el train en lugar de generalizar. Señales en nuestro entrenamiento:

- Época 7: train_loss=0.86, val_loss=1.12 → equilibrio razonable
- Época 8: train_loss=0.84, val_loss=**1.31** → val_loss subió 16,9% mientras train bajaba

Decisión: guardamos el checkpoint de la **época 7** como **best_model.pt**.

4.8. Mixed Precision (AMP)

Entrenar con **float16** en la GPU en lugar de float32. Acelera ~2× en Tesla T4 sin perder calidad perceptible. PyTorch lo maneja con **autocast** y **GradScaler**.

4.9. Learning Rate Scheduler

ReduceLROnPlateau: si el AUC en validación no mejora durante 2 épocas, el learning rate se multiplica por 0.1 (de 1e-4 → 1e-5 → 1e-6). Ayuda a salir de mesetas de entrenamiento.

5. DenseNet-121 y CheXNet en detalle

¿Qué es DenseNet?

DenseNet (Densely Connected Convolutional Networks) es una arquitectura de CNN propuesta por Huang et al. (2017). Su idea central son las **conexiones densas**:

Cada capa recibe como entrada **todas las salidas de las capas anteriores**, no solo la inmediata anterior.



Ventajas:

1. **Reutilización de características:** patrones detectados en capas tempranas (bordes, texturas) se propagan directamente a capas profundas.
2. **Gradientes más estables:** las conexiones densas facilitan el backpropagation en redes profundas.
3. **Menos parámetros redundantes:** no necesita reaprender las mismas características en cada capa.
4. **Ideal para patologías correlacionadas:** como Edema co-ocurre con Infiltration (43%), compartir representaciones ayuda.

¿Qué significa el "121"?

El número indica la **profundidad** de la red: 121 capas con parámetros entrenables. Es un balance entre capacidad y eficiencia computacional.

Modelo	Capas	Parámetros aprox.	Uso típico
DenseNet-121	121	~8 millones	Nuestro proyecto, CheXNet
DenseNet-169	169	~14 millones	Más preciso, más lento
ResNet-50	50	~25 millones	Alternativa común
EfficientNet-B4	—	~19 millones	Más moderno

Estructura interna de DenseNet-121 (simplificada)

```

Entrada: imagen 3x224x224
|
▼
Conv inicial + MaxPool
|
▼
Dense Block 1 (6 capas densas)
|
▼
Transition Layer 1 (reduce canales, downsampling)
|
▼
Dense Block 2 (12 capas densas)
|
▼
Transition Layer 2
|
▼
Dense Block 3 (24 capas densas)
|
▼
Transition Layer 3
|
▼
Dense Block 4 (16 capas densas) ← Grad-CAM usa esta capa
|
▼
Global Average Pooling → vector de 1024 features
|
▼
Linear(1024 → 14) → 14 logits (nuestra cabeza multi-etiqueta)

```

¿Qué es CheXNet?

CheXNet (Rajpurkar et al., Stanford 2017) es el paper de referencia que aplicó DenseNet-121 al dataset NIH ChestX-ray14. Logró un AUC promedio de **~0.84** en las 14 patologías, comparable a radiólogos en algunas clases.

Nuestro modelo está inspirado en CheXNet:

- Misma arquitectura: DenseNet-121
- Mismo dataset: NIH ChestX-ray14
- Mismo tipo de problema: multi-etiqueta con sigmoid + BCE
- Mismo tamaño de imagen: 224×224

Diferencias con CheXNet original:

Aspecto	CheXNet (2017)	Nuestro modelo
Épocas	~80	10
GPU	NVIDIA TitanX (múltiples)	Tesla T4 (Kaggle)
Augmentation	Incluye flip horizontal	Sin flip horizontal
Test AUC promedio	0.8399	0.7938

6. Transfer Learning explicado paso a paso

Definición

Transfer Learning (aprendizaje por transferencia) es reutilizar un modelo ya entrenado en un problema grande (dominio fuente) para resolver un problema relacionado pero diferente (dominio destino), ajustando solo lo necesario.

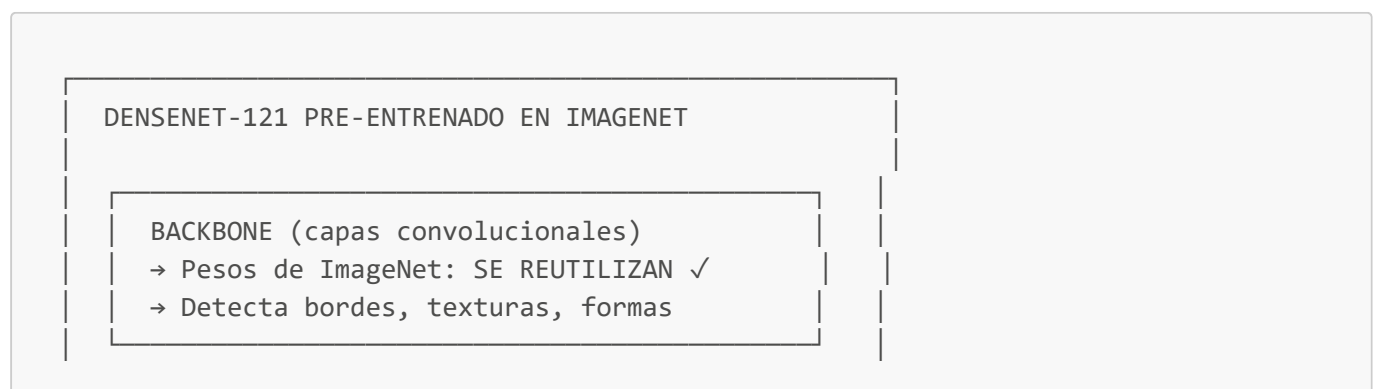
Analogía simple

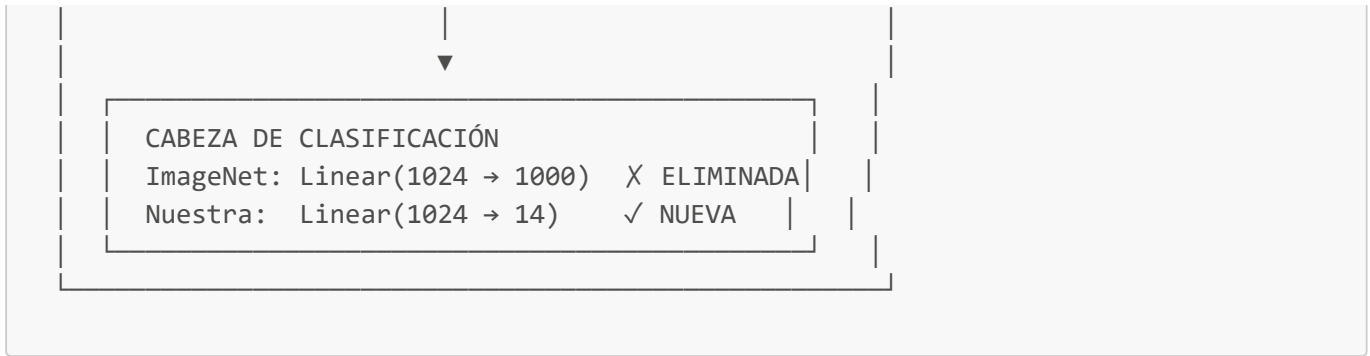
Imagina que alguien ya sabe **reconocer formas, bordes, texturas y objetos** en millones de fotos de internet (ImageNet). Tú le enseñas a reconocer **patologías en radiografías** aprovechando ese conocimiento previo, en lugar de empezar desde cero.

En nuestro proyecto

	Dominio fuente	Dominio destino
Dataset	ImageNet (~1,2 millones de fotos naturales)	NIH ChestX-ray14 (112.120 radiografías)
Tarea	Clasificar 1000 clases (gatos, autos, etc.)	Detectar 14 patologías torácicas
Modelo base	DenseNet-121 pre-entrenado	Mismo backbone, cabeza nueva

¿Qué se transfiere y qué se reemplaza?





¿Por qué funciona con radiografías si ImageNet son fotos naturales?

Las capas **tempranas** de una CNN aprenden features **genéricas** (bordes, gradientes, texturas) que sirven para cualquier imagen. Las capas **profundas** aprenden features específicas del dominio, y esas se **re-entrenan** con nuestras radiografías.

Normalización ImageNet

Aunque nuestras imágenes son radiografías (no fotos de ImageNet), normalizamos con las estadísticas de ImageNet:

```
mean = [0.485, 0.456, 0.406]
std  = [0.229, 0.224, 0.225]
```

Razón: los pesos pre-entrenados esperan entradas con esa distribución. Cambiar la normalización desalinearía las activaciones internas.

Replicar canal de grises a RGB

Las radiografías son **monocromáticas** (1 canal). ImageNet usa **3 canales** (RGB). Solución:

```
T.Grayscale(num_output_channels=3) # 1 canal → 3 canales idénticos
```

Ventajas del Transfer Learning en medicina

1. **Menos datos necesarios:** 112K imágenes médicas son pocas comparadas con ImageNet; transferir acelera el aprendizaje.
2. **Menos tiempo de entrenamiento:** 10 épocas nos dieron AUC 0.79; desde cero hubiera sido mucho peor.
3. **Mejor generalización:** los filtros de bajo nivel ya están bien calibrados.

7. Fases del proyecto (qué hicimos y en qué orden)

[Fase 1] EDA local	→ Entender el dataset (metadata, sin GPU)
[Fase 2] Diseño	→ Scripts de splits, dataset, modelo, transforms

[Fase 3] Entrenamiento	→ Kaggle GPU (10 épocas, ~4.2 horas)
[Fase 4] Evaluación	→ Métricas en test, comparación con CheXNet
[Fase 5] Implementación	→ App Streamlit + Grad-CAM

Fase	Objetivo específico	Entregable principal
Análisis	4.2.1 — Analizar características del dataset	<code>docs/01_fase_analisis.md</code> + figuras EDA
Diseño	4.2.2 — Diseñar arquitectura y pipeline	<code>docs/02_fase_diseno.md</code> + scripts <code>src/</code>
Entrenamiento	4.2.4 — Entrenar y validar el modelo	<code>best_model.pt</code> , <code>history.json</code>
Evaluación	4.2.5 — Analizar efectividad	<code>docs/03_fase_prueba.md</code> , <code>test_results.json</code>
Implementación	4.2.3 — Demo funcional	<code>app.py</code> (Streamlit PulmoScan)

8. Fase 1: Análisis Exploratorio (EDA)

Archivo: `notebooks/01_eda_metadata.ipynb`

Documento: `docs/01_fase_analisis.md`

Qué hicimos

Analizamos el CSV de metadatos (9,4 MB, cabe en RAM) **sin necesitar las imágenes**. Generamos 7 figuras en `outputs/figures/`.

Hallazgos principales y decisiones que tomamos

#	Hallazgo	Decisión de diseño
1	0 valores nulos en todo el CSV	No hay limpieza de NaN
2	Imágenes con 0 a 9 patologías simultáneas	Clasificación multi-etiqueta (sigmoid + BCE)
3	Infiltration 87× más frecuente que Hernia	<code>pos_weight</code> en la loss; AUC-ROC en lugar de accuracy
4	Patologías correlacionadas (Edema+Infiltration 43%)	DenseNet-121 con representaciones compartidas
5	Pacientes con hasta 184 imágenes	Split por <code>patient_id</code> , nunca por imagen
6	Test enriquecido en enfermos (prevalencias mayores)	Documentar caída de AUC en test; usar AUC-ROC
7	Resolución promedio 2.646×2.486 px	Redimensionar a 224×224
8	53,8% imágenes "No Finding"	14 salidas (No Finding implícito)

Figuras generadas

Figura	Archivo	Qué muestra
1	01_distribucion_patologias.png	Frecuencia de cada patología
2	02_no_finding_vs_finding.png	Sano vs con hallazgo
3	03_hallazgos_por_imagen.png	Cuántas patologías por radiografía
4	04_cooccurrence.png	Matriz de co-ocurrencia condicional
5	05_demografia.png	Edad, género, vista PA/AP
6	06_prevalencia_por_edad.png	Prevalencia por grupo etario
7	07_imgs_por_paciente.png	Imágenes por paciente (leakage)

Codificación multi-etiqueta

Transformamos texto como "[Cardiomegaly|Effusion](#)" en un vector binario:

```
Atelectasis  Cardiomegaly  Effusion  Infiltration  ...  Hernia
0            1            1            0            ...      0
```

Esto se guardó en [data/processed/metadata_clean.csv](#) (112.120 filas × 23 columnas).

9. Fase 2: Diseño del sistema

Documento: [docs/02_fase_diseno.md](#)

9.1. [src/data/build_splits.py](#) — División train/val/test

Qué hace:

1. Lee [metadata_clean.csv](#)
2. Asigna [test](#) a las imágenes de [test_list.txt](#) (oficial NIH)
3. Sobre el resto ([train_val](#)), aplica **GroupKFold** agrupando por [patient_id](#)
4. El fold 0 (configurable) se usa como validación
5. Verifica que no haya pacientes compartidos entre splits

Comando:

```
python -m src.data.build_splits --fold 0 --n-splits 5 --seed 42
```

Salida: [data/processed/metadata_with_splits.csv](#)

9.2. [src/data/transforms.py](#) — Preprocesamiento

Pipeline de transformaciones con [torchvision.transforms](#):

```
PIL Image (grayscale)
→ Grayscale(3 canales)
→ Resize(224×224)
→ [Train only: RandomAffine + ColorJitter]
→ ToTensor()
→ Normalize(ImageNet mean/std)
→ Tensor (3, 224, 224)
```

9.3. `src/data/dataset.py` — Dataset PyTorch

Clase `ChestXrayDataset(Dataset)`:

- **Lazy loading:** abre cada PNG solo cuando el DataLoader lo pide
- **Output:** (`imagen_tensor`, `label_vector_14`) donde label es float32 con 0.0/1.0
- **Robustez:** si una imagen falta, devuelve imagen negra + warning
- **return_meta=True:** devuelve además patient_id, age, gender, view
- **compute_pos_weight():** calcula los pesos para BCEWithLogitsLoss

9.4. `src/models/densenet_chexnet.py` — El modelo

Clase `DenseNetCheXNet(nn.Module)`:

```
# Pseudocódigo simplificado
backbone = densenet121(weights=IMAGENET1K_V1) # Transfer Learning
backbone.classifier = Linear(1024, 14)         # Cabeza nueva
# forward() → logits (14,) sin sigmoid
# predict_proba() → sigmoid(logits) para inferencia
```

- Inicialización Xavier de la cabeza nueva
- ~7,98 millones de parámetros totales
- **feature_extractor** expone las capas convolucionales para Grad-CAM

9.5. `src/data/extract_metadata.py` — Utilidad

Extrae solo los CSV y listas de texto del ZIP de 45 GB, sin descomprimir las imágenes. Útil para trabajar en local.

10. Fase 3: Entrenamiento en Kaggle

Archivo: `notebooks/02_kaggle_training.ipynb`

¿Por qué Kaggle y no la PC local?

La GPU local (AMD Radeon RX 580) **no soporta CUDA** en Windows. PyTorch necesita CUDA para entrenar en GPU NVIDIA. Kaggle ofrece **Tesla T4 gratis** (~30 h/semana) con el dataset NIH ya montado.

Configuración en Kaggle

1. Crear notebook → Settings → Accelerator → **GPU T4 x1**
2. Add Data → [nih-chest-xrays/data](#) (imágenes)
3. Add Data → dataset privado con [metadata_with_splits.csv](#)

Hiperparámetros

Parámetro	Valor
Arquitectura	DenseNet-121 (ImageNet weights)
Imagen	224 × 224 px
Batch size	32
Épocas	10
Optimizer	Adam (lr=1e-4, weight_decay=1e-5)
Loss	BCEWithLogitsLoss(pos_weight=...)
Scheduler	ReduceLROnPlateau (factor=0.1, patience=2)
Mixed precision	AMP (float16)
Semilla	42

Loop de entrenamiento (conceptual)

Para cada época:

1. Modo train → forward + backward + update weights
2. Modo eval → calcular val_loss y val_AUC (sin gradientes)
3. Si val_AUC > best_AUC → guardar best_model.pt
4. Scheduler.step(val_AUC)

Archivos generados al entrenar

Archivo	Contenido
best_model.pt	Pesos del mejor checkpoint (época 7)
history.json	Loss y AUC por época
test_results.json	AUC, F1, AP por clase en test
test_predictions.npz	Probabilidades, labels, image_ids
training_curves.png	Gráficas de convergencia

11. Fase 4: Evaluación y resultados

Documento: [docs/03_fase_prueba.md](#)

Notebook: [notebooks/03_evaluacion_resultados.ipynb](#)

Métricas que usamos

Métrica	Qué mide	Por qué la usamos
AUC-ROC	Capacidad de separar positivos de negativos	Insensible al desbalance; estándar en CheXNet
F1-Score	Balance precisión/recall	Útil clínicamente por clase
Average Precision (AP)	Área bajo curva Precision-Recall	No depende de umbral fijo
Loss (BCE)	Error de entrenamiento	Detectar overfitting

Métricas que NO usamos

- **Accuracy global:** un modelo que siempre dice "no" tendría ~99% accuracy en Hernia pero es inútil.

Curvas de aprendizaje (10 épocas)

Época	Train Loss	Val Loss	Val AUC	Nota
1	1.1714	1.1142	0.7817	Aprendizaje rápido
2	1.0535	1.0637	0.8007	+2.4 pts AUC
5	0.9275	1.1000	0.8076	Refinamiento
6	0.8892	1.1320	0.8153	
7	0.8606	1.1202	0.8178	← MEJOR CHECKPOINT
8	0.8369	1.3105	0.8054	Overfitting: val_loss +16.9%
10	0.7762	1.2473	0.8137	LR reducido a 1e-5

Resultados finales en test

Métrica	Valor
Test AUC-ROC promedio	0.7938
Val AUC (mejor época)	0.8178
Test Loss	1.5716
Mejor época	7
Imágenes evaluadas	25.596

AUC por patología (test)

Patología	Nuestro AUC	CheXNet AUC	Diferencia
Hernia	0.9122	0.9164	-0.004

Patología	Nuestro AUC	CheXNet AUC	Diferencia
Cardiomegaly	0.8879	0.9248	-0.037
Emphysema	0.8864	0.9371	-0.051
Pneumothorax	0.8377	0.8887	-0.051
Edema	0.8362	0.8878	-0.052
Fibrosis	0.8218	0.8047	+0.017 ✓
Effusion	0.8138	0.8638	-0.050
Mass	0.7948	0.8676	-0.073
Pleural_Thickening	0.7541	0.7856	-0.032
Atelectasis	0.7439	0.8094	-0.066
Consolidation	0.7379	0.7901	-0.052
Nodule	0.7214	0.7802	-0.059
Infiltration	0.6883	0.7345	-0.046
Pneumonia	0.6770	0.7680	-0.091
PROMEDIO	0.7938	0.8399	-0.046

¿Por qué test AUC < val AUC?

Es esperado. El test set del NIH tiene prevalencias mucho más altas de enfermedad:

Patología	Prevalencia train	Prevalencia test	Factor
Pneumothorax	3,0%	10,4%	×3,5
Effusion	10,0%	18,2%	×1,8
Infiltration	16,0%	22,6%	×1,5

El NIH diseñó el test así a propósito para que sea clínicamente más exigente.

Logros destacables

1. **Fibrosis supera a CheXNet** (+0.017 AUC) — única clase donde lo superamos
2. **Hernia casi iguala a CheXNet** (-0.004) — a pesar de ser la clase más rara
3. A 10 épocas estamos a solo 0.046 del paper original entrenado ~80 épocas

12. Fase 5: App de demostración (Streamlit)

Archivo principal: `app.py`

UI: carpeta `ui/`

PulmoScan — páginas de la app

Página	Archivo	Función
Análisis de imagen	ui/pages/analisis.py	Subir radiografía → predicciones + Grad-CAM
Exploración del dataset	ui/pages/eda.py	Muestra las 7 figuras del EDA
Evaluación del modelo	ui/pages/evaluacion.py	Métricas, curvas ROC, comparación CheXNet
Acerca del proyecto	ui/pages/acerca.py	Info del equipo, disclaimer, referencias

Flujo de inferencia

```

Usuario sube PNG
→ get_eval_transforms() (resize 224, normalizar)
→ model(tensor) → logits
→ sigmoid → 14 probabilidades
→ Umbral (default 0.30) → patologías detectadas
→ Grad-CAM → mapa de calor sobre la imagen
    
```

Componentes clave

- [src/inference/predictor.py](#): carga [best_model.pt](#), ejecuta predicción
- [src/inference/gradcam.py](#): genera mapas de atención
- [src/config.py](#): constantes (patologías, AUC de CheXNet, figuras EDA)
- [ui/theme.py](#): estilos CSS personalizados

Cómo ejecutar la app

```

# Con el entorno virtual activado
streamlit run app.py
    
```

Requisito: tener [outputs/models/best_model.pt](#) descargado de Kaggle.

13. Grad-CAM: interpretabilidad del modelo

¿Qué es Grad-CAM?

Gradient-weighted Class Activation Mapping es una técnica para visualizar **en qué regiones de la imagen** el modelo se fijó para tomar su decisión. Genera un mapa de calor superpuesto sobre la radiografía.

¿Por qué importa en medicina?

Un modelo de caja negra que dice "Pneumonia 87%" sin explicar dónde ve la neumonía no es confiable. Grad-CAM permite al usuario (o radiólogo) verificar si el modelo mira la región correcta del pulmón.

Cómo funciona (simplificado)

1. Pasar la imagen por el modelo
2. Calcular el gradiente de la clase objetivo (ej. Pneumonia) respecto al último mapa de activaciones (`denseblock4`)
3. Promediar los gradientes → pesos de importancia por canal
4. Combinar pesos × activaciones → mapa de calor
5. Superponer sobre la imagen original

Implementación en nuestro proyecto

Archivo: `src/inference/gradcam.py`

- Capa objetivo: `model.backbone.features.denseblock4`
- Validación futura: comparar con `BBox_List_2017.csv` (984 bounding boxes del NIH)

14. Mapa de archivos del repositorio

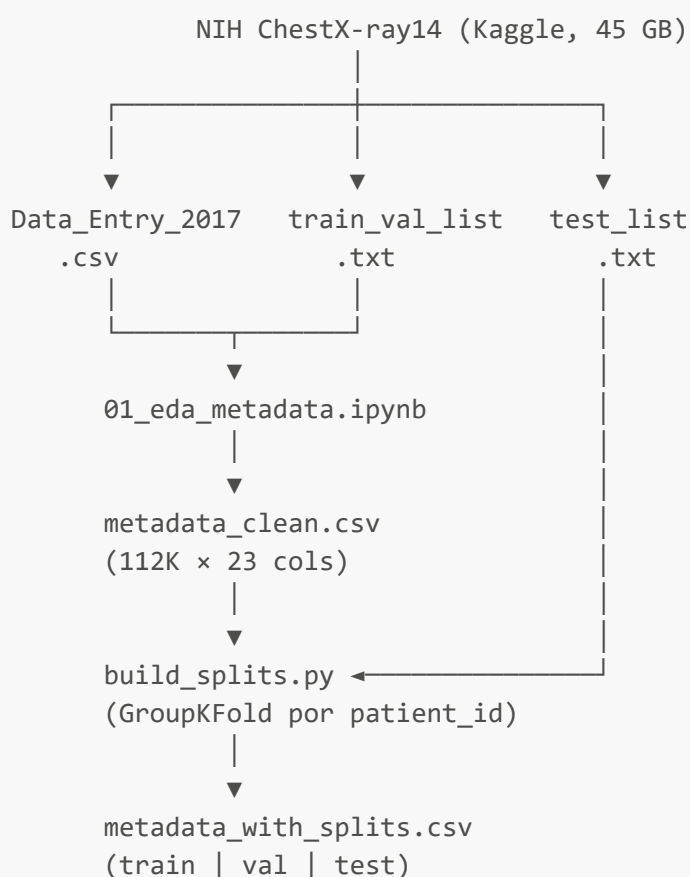
```

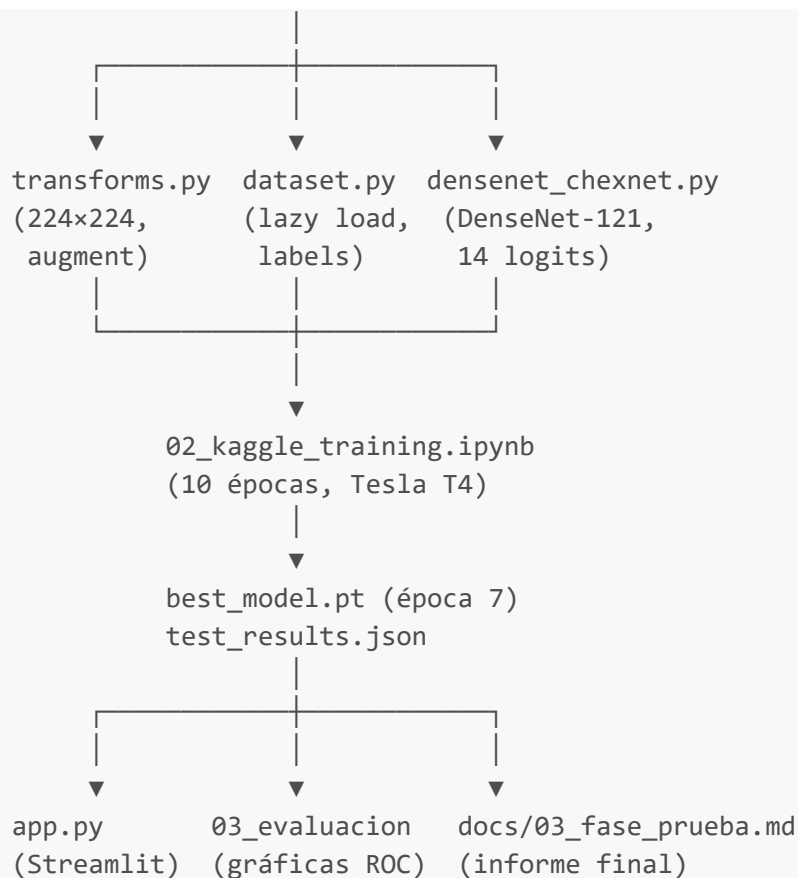
proyecto_fase3/
├── app.py                ← App Streamlit (PulmoScan)
├── requirements.txt      ← Dependencias Python
├── md_to_pdf.py         ← Convierte .md a PDF
├── data/
│   ├── raw/             ← CSVs del NIH (sin imágenes)
│   │   ├── Data_Entry_2017.csv
│   │   ├── train_val_list.txt
│   │   ├── test_list.txt
│   │   └── BBox_List_2017.csv
│   ├── processed/
│   │   ├── metadata_clean.csv    ← EDA: 23 columnas + labels binarios
│   │   └── metadata_with_splits.csv ← Splits train/val/test
│   └── sample_images/          ← Radiografías de ejemplo para la demo
├── docs/
│   ├── GUIA_ESTUDIO_COMPLETA.md ← ESTE DOCUMENTO
│   ├── 01_fase_analisis.md      ← Informe fase de análisis
│   ├── 02_fase_diseno.md        ← Informe fase de diseño
│   └── 03_fase_prueba.md        ← Informe fase de prueba
├── notebooks/
│   ├── 01_eda_metadata.ipynb    ← EDA (ejecutar local, CPU)
│   ├── 02_kaggle_training.ipynb ← Entrenamiento (ejecutar en Kaggle, GPU)
│   └── 03_evaluacion_resultados.ipynb ← Evaluación y gráficas
├── outputs/
│   ├── figures/                ← 12 figuras (EDA + evaluación)
│   └── models/
│       ├── best_model.pt       ← Pesos del modelo (época 7)
│       ├── history.json        ← Loss/AUC por época
│       └── test_results.json   ← Métricas finales en test

```

src/	
├─ config.py	← Constantes globales
├─ data/	
│ ├─ build_splits.py	← Genera train/val/test
│ ├─ dataset.py	← ChestXrayDataset (PyTorch)
│ ├─ transforms.py	← Preprocesamiento + augmentation
│ └─ extract_metadata.py	← Extrae CSVs del ZIP
├─ models/	
│ └─ densenet_chexnet.py	← DenseNet-121 multi-etiqueta
├─ inference/	
│ ├─ predictor.py	← Carga modelo + predice
│ └─ gradcam.py	← Mapas de atención
└─ reports/	
└─ pdf_builder.py	← Generador de PDFs
ui/	
├─ theme.py	← CSS y estilos
├─ components.py	← Componentes reutilizables
└─ pages/	
├─ analisis.py	← Página de predicción
├─ eda.py	← Página de exploración
├─ evaluacion.py	← Página de métricas
└─ acerca.py	← Página informativa

15. Flujo de datos completo (diagrama)





16. Resultados numéricos clave (para memorizar)

Dato	Valor
Dataset total	112.120 imágenes, 30.805 pacientes
Patologías	14 clases multi-etiqueta
Arquitectura	DenseNet-121 (~8M parámetros)
Transfer Learning	ImageNet → NIH ChestX-ray14
Mejor época	7 de 10
Val AUC (mejor)	0.8178
Test AUC (final)	0.7938
CheXNet AUC (ref.)	0.8399
Mejor clase	Hernia (AUC 0.9122)
Peor clase	Pneumonia (AUC 0.6770)
Única clase > CheXNet	Fibrosis (+0.017)
Batch size	32
Learning rate	1e-4 (Adam)

Dato	Valor
Tiempo entrenamiento	~4.2 horas (Kaggle T4)
Umbral app	0.30 (configurable)

17. Preguntas frecuentes para sustentación

"¿Por qué multi-etiqueta y no clasificación normal?"

Porque el 22% de las radiografías con hallazgos tienen 2+ patologías simultáneas. Softmax obligaría a elegir una sola; sigmoid permite múltiples.

"¿Qué es Transfer Learning y por qué lo usaron?"

Reutilizar DenseNet-121 pre-entrenado en ImageNet (1,2M fotos) para no empezar desde cero con solo 112K radiografías. Se reemplaza la cabeza (1000→14 clases) y se re-entrena.

"¿Qué es DenseNet-121?"

CNN de 121 capas con conexiones densas (cada capa recibe todas las anteriores). Reutiliza características eficientemente. Es la arquitectura de CheXNet.

"¿Cómo evitaron el data leakage?"

Dividiendo por `patient_id` con GroupKFold, nunca por imagen. Verificaron intersección 0 entre splits. Respetaron splits oficiales del NIH.

"¿Por qué AUC-ROC y no accuracy?"

Porque con desbalance extremo (Hernia 0,2% vs Infiltration 17,7%), un modelo que siempre dice "no" tiene accuracy altísima pero es inútil. AUC mide separabilidad independiente de prevalencia.

"¿Qué es pos_weight?"

Peso que penaliza más los falsos negativos en clases raras. Hernia tiene `pos_weight`≈626, forzando al modelo a aprenderla.

"¿Por qué el test AUC es menor que val AUC?"

El test set del NIH tiene prevalencias de enfermedad mucho más altas (deliberadamente). No es overfitting; es un escenario clínico más difícil.

"¿Por qué entrenaron en Kaggle?"

La GPU local (AMD RX 580) no soporta CUDA. Kaggle ofrece Tesla T4 gratis con el dataset ya montado.

"¿Qué es Grad-CAM?"

Técnica de interpretabilidad que genera un mapa de calor mostrando dónde el modelo "miró" en la imagen para cada predicción.

"¿Por qué no usaron flip horizontal?"

Porque invertir la imagen cambiaría la lateralidad cardíaca (corazón a la izquierda), afectando la detección de Cardiomegaly.

"¿Cuál fue el mayor logro?"

Fibrosis superó a CheXNet (+0.017 AUC) y el promedio general (0.7938) está a solo 0.046 del paper original entrenado 8× más épocas.

18. Glosario

Término	Definición
AUC-ROC	Area Under the Curve — Receiver Operating Characteristic. Mide qué tan bien separa positivos de negativos. 1.0 = perfecto, 0.5 = azar.
Backbone	Parte convolucional de la red (sin la cabeza de clasificación). En nuestro caso: DenseNet-121 features.
BCE	Binary Cross-Entropy. Función de pérdida para clasificación binaria/multi-etiqueta.
Batch size	Número de imágenes procesadas simultáneamente en GPU. Nosotros: 32.
Checkpoint	Archivo con los pesos del modelo en un punto del entrenamiento. Guardamos el de la época 7.
CheXNet	Modelo de Stanford (2017) que aplicó DenseNet-121 a NIH ChestX-ray14. Nuestra referencia.
CNN	Convolutional Neural Network. Red neuronal para imágenes.
CUDA	Plataforma de NVIDIA para computación en GPU. Requerida por PyTorch para entrenar en GPU.
Data Augmentation	Transformaciones artísticas de imágenes de entrenamiento para mejorar generalización.
Data Leakage	Cuando información de test contamina el entrenamiento, inflando métricas.
DenseNet	Arquitectura CNN con conexiones densas entre capas.
Dropout	Técnica de regularización que apaga neuronas aleatoriamente. No la usamos (0.0).
Época (epoch)	Una pasada completa por todo el dataset de entrenamiento.
F1-Score	Media armónica de precisión y recall.
Grad-CAM	Gradient-weighted Class Activation Mapping. Mapa de calor de atención del modelo.
GroupKFold	Validación cruzada que respeta grupos (pacientes). Evita leakage.
ImageNet	Dataset de 1,2M imágenes naturales usado para pre-entrenar modelos.

Término	Definición
Lazy loading	Cargar datos bajo demanda (al vuelo) en lugar de precargar todo en RAM.
Logits	Salidas crudas de la red antes de aplicar sigmoid/softmax.
Mixed Precision (AMP)	Entrenar con float16 para acelerar en GPU.
Multi-etiqueta	Cada muestra puede tener múltiples clases positivas simultáneamente.
Overfitting	Modelo memoriza train y no generaliza a datos nuevos.
pos_weight	Peso para compensar desbalance en BCEWithLogitsLoss.
Sigmoid	Función que mapea cualquier número a (0, 1). Una por cada clase.
Softmax	Función que convierte logits en probabilidades que suman 1. NO la usamos.
Transfer Learning	Reutilizar un modelo pre-entrenado en otro problema relacionado.
Umbral (threshold)	Probabilidad mínima para considerar positiva una clase. App: 0.30.

Referencias bibliográficas

1. **NIH ChestX-ray14:** Wang, X. et al. (2017). "ChestX-ray8: Hospital-scale Chest X-ray Database and Benchmarks on Weakly-Supervised Classification and Localization of Common Thorax Diseases." CVPR.
2. **CheXNet:** Rajpurkar, P. et al. (2017). "CheXNet: Radiologist-Level Pneumonia Detection on Chest X-Rays with Deep Learning." arXiv:1711.05225.
3. **DenseNet:** Huang, G. et al. (2017). "Densely Connected Convolutional Networks." CVPR.
4. **Grad-CAM:** Selvaraju, R. et al. (2017). "Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization." ICCV.

Documento generado para estudio y sustentación del proyecto PulmoScan — UPC 2026-I.